Program 11: Top-Down Parser for Context-Free Grammar

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the'
N -> 'boy' | 'apple'
V -> 'eats'
""")
parser = nltk.RecursiveDescentParser(grammar)

sentence = "the boy eats the apple".split()

print("Input Sentence:", " ".join(sentence))
print("Parse Tree:")

for tree in parser.parse(sentence):
    print(tree)
```

```
Input Sentence: the boy eats the apple
Parse Tree:
(S (NP (Det the) (N boy)) (VP (V eats) (NP (Det the) (N apple))))
```

Program 12: Earley Parser for Context-Free Grammar

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'cat' | 'dog' | 'fish'
V -> 'eats' | 'likes'
""")
parser = nltk.EarleyChartParser(grammar)

sentence = "the cat eats a fish".split()

print("Input Sentence:", " ".join(sentence))
print("Earley Parse:")

for tree in parser.parse(sentence):
    print(tree)
```

```
Input Sentence: the cat eats a fish
Earley Parse:
(S (NP (Det the) (N cat)) (VP (V eats) (NP (Det a) (N fish))))
```

Program 13: Generate a Parse Tree for a Sentence

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> Det Adj N
VP -> V NP
Det -> 'the'
Adj -> 'small'
N -> 'boy' | 'ball'
V -> 'kicks'
""")
parser = nltk.ChartParser(grammar)
sentence = "the small boy kicks the small ball".split()
for tree in parser.parse(sentence):
    print("Parse Tree:")
    print(tree)
    tree.pretty_print()
```

```
Parse Tree:
(S
  (NP (Det the) (Adj small) (N boy))
  (VP (V kicks) (NP (Det the) (Adj small) (N ball))))
                  S
      ____________|_________
     |                     VP
     |                  ____|____
     NP                |         NP
  ___|____             |      ___|____
 Det  Adj   N     V    Det    Adj    N
  |    |    |     |     |      |     |
 the small boy kicks the    small ball
```

Program 14: Agreement Checking in Sentences

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP_S VP_S
S -> NP_P VP_P
NP_S -> 'he' | 'she' | 'the boy'
NP_P -> 'they' | 'we' | 'the boys'
VP_S -> 'plays' | 'runs'
VP_P -> 'play' | 'run'
""")
parser = nltk.ChartParser(grammar)
sentences = [
    "he plays",
    "they play",
    "he play",
    "they plays"
]
for sentence in sentences:
    words = sentence.split()
    trees = list(parser.parse(words))
    if trees:
        print(sentence, "-> Correct Agreement")
    else:
        print(sentence, "-> Incorrect Agreement")
```

```
he plays -> Correct Agreement
they play -> Correct Agreement
he play -> Incorrect Agreement
they plays -> Incorrect Agreement
```

Program 15: Probabilistic Context-Free Grammar (PCFG) Parsing

```
import nltk

grammar = nltk.PCFG.fromstring("""
S -> NP VP [1.0]
NP -> Det N [0.8] | N [0.2]
VP -> V NP [0.7] | V [0.3]
Det -> 'the' [0.5] | 'a' [0.5]
N -> 'dog' [0.5] | 'cat' [0.5]
V -> 'sees' [1.0]
""")

parser = nltk.ViterbiParser(grammar)

sentence = "the dog sees a cat".split()

print("Input Sentence:", " ".join(sentence))
print("Probabilistic Parse:")

for tree in parser.parse(sentence):
    print(tree)
    print("Probability:", tree.prob())
```

```
Input Sentence: the dog sees a cat
Probabilistic Parse:
(S
  (NP (Det the) (N dog))
  (VP (V sees) (NP (Det a) (N cat)))) (p=0.028)
Probability: 0.027999999999999997
```

Program 16: Named Entity Recognition Using SpaCy

```
import spacy

# Load English language model
nlp = spacy.load("en_core_web_sm")

text = "Rahul lives in Chennai and studies at Google University."

# Process the text
doc = nlp(text)

print("Named Entities:")

for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

```
Named Entities:
Chennai -> ORG
Google University -> ORG
```

Program 17: Access WordNet and Retrieve Synsets

```python
from nltk.corpus import wordnet

word = "bank"

synsets = wordnet.synsets(word)

print("Word:", word)
print("\nSynsets and Meanings:")

for syn in synsets[:5]:
    print("\nSynset:", syn.name())
    print("Meaning:", syn.definition())
```

```
Word: bank

Synsets and Meanings:

Synset: bank.n.01
Meaning: sloping land (especially the slope beside a body of water)

Synset: depository_financial_institution.n.01
Meaning: a financial institution that accepts deposits and channels the money into lending activities

Synset: bank.n.03
Meaning: a long ridge or pile

Synset: bank.n.04
Meaning: an arrangement of similar objects in a row or in tiers

Synset: bank.n.05
Meaning: a supply or stock held in reserve for future use (especially in emergencies)
```

Program 18: First-Order Predicate Logic (FOPC) Parser

```python
import re
tokens = re.compile(
    r'forall|exists|->|[A-Za-z_][A-Za-z0-9_]*|[(),&|~]'
)
def parse(expression):
    words = tokens.findall(expression)
    result = " ".join(words)
    if result.replace(" ", "") != expression.replace(" ", ""):
        return False
    pattern = re.compile(
        r'^(forall|exists)\s+[a-z]\s+.+$|'
        r'^[A-Za-z_]\w*\s*\([^()]*\)$|'
        r'^~\s*[A-Za-z_]\w*\s*\([^()]*\)$'
    )
    return bool(pattern.match(expression.strip()))


expressions = [
    "forall x P(x)",
    "exists y Loves(y, John)",
    "~Human(x)",
    "P(x) & Q(x)"
]

for exp in expressions:
    print(exp, "->", "Valid" if parse(exp) else "Invalid")
```

```
forall x P(x) -> Valid
exists y Loves(y, John) -> Valid
~Human(x) -> Valid
P(x) & Q(x) -> Invalid
```

Corrected Program 18: FOPC Parser

```
import re

def check(expression):
    pattern = r'^(forall|exists)\s+[a-z]\s+[A-Za-z]+\([a-zA-Z]+\)$
    if re.match(pattern, expression):
        return "Valid FOPC Expression"
    else:
        return "Invalid FOPC Expression"

exp = input("Enter logical expression: ")

print(check(exp))
```

```
Enter logical expression: forall x Human(x)
Valid FOPC Expression
```

Program 19: Word Sense Disambiguation Using Lesk Algorithm

```
from nltk.wsd import lesk

sentence = "I went to the bank to deposit money"

word = "bank"

# Apply Lesk algorithm
result = lesk(sentence.split(), word, "n")

print("Sentence:", sentence)
print("Ambiguous Word:", word)

if result:
    print("Selected Meaning:", result.definition())
    print("Synset:", result.name())
else:
    print("Meaning not found")
```

```
Sentence: I went to the bank to deposit money
Ambiguous Word: bank
Selected Meaning: a container (usually with a slot in the top) for keeping money at home
Synset: savings_bank.n.02
```

Program 20: Information Retrieval Using TF-IDF

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Documents
documents = [
    "Python is a programming language",
    "Python is used in machine learning",
    "Natural language processing uses Python",
    "Java is another programming language"
]

# User search query
query = "Python programming language"

# Convert documents into TF-IDF vectors
vectorizer = TfidfVectorizer()

doc_vectors = vectorizer.fit_transform(documents)

# Convert query into TF-IDF vector
query_vector = vectorizer.transform([query])

# Calculate similarity scores
scores = cosine_similarity(query_vector, doc_vectors)[0]

# Rank documents
ranking = scores.argsort()[::-1]

print("Search Query:", query)
print("\nDocument Ranking:")

for i in ranking:
    print("Document", i + 1, "Score:", round(scores[i], 3))
    print(documents[i])
```

```
Search Query: Python programming language

Document Ranking:
Document 1 Score: 0.883
Python is a programming language
Document 4 Score: 0.463
Java is another programming language
Document 3 Score: 0.348
Natural language processing uses Python
Document 2 Score: 0.155
Python is used in machine learning
```